

Chapter Three

Security Techniques

Part – III

Stream Block Cipher
DES and AES

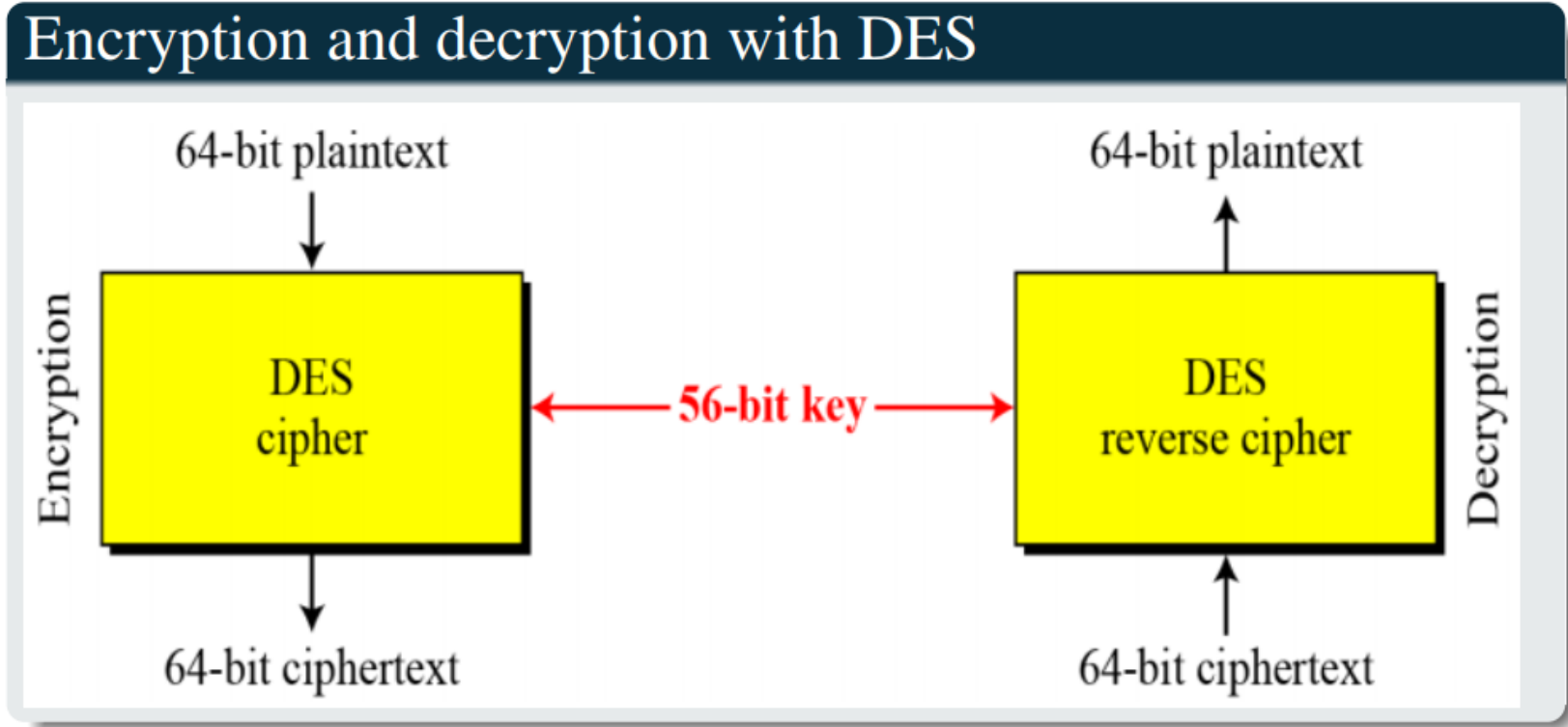
Information Security
(SE7432)

DATA ENCRYPTION STANDARD (DES)

- ✧ The Data Encryption Standard (DES) was developed in the 1970s by the National Bureau of Standards with the help of the NSA
- ✧ Its purpose is to provide a standard method for protecting sensitive commercial and unclassified data
- ✧ DES officially became a federal standard in November of 1976 as the standard encryption algorithm for government applications
- ✧ DES is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST)
- ✧ DES is a careful and complex combination of two fundamental building blocks of encryption: Substitution and Transposition
- ✧ DES uses 56-bit encryption key to operate on 64-bit blocks of data
- ✧ The 56-bit key is divided into eight 7-bit blocks and an 8th odd parity bit is added to each block

DATA ENCRYPTION STANDARD (DES)

- ✧ By using the 8 parity bits for rudimentary error detection, a DES key is actually 64 bits in length for computational purposes



DES OPERATION

- ✧ The encryption process is made of two permutations (P-boxes) which are initial & final permutations, and sixteen Feistel rounds
- ✧ Permutation is an operation performed by a function, which moves an element at place j to the place k
- ✧ The P-boxes provide diffusion across S-box inputs
- ✧ The S-boxes provide confusion of input bits
- ✧ Confusion: A technique that seeks to make the relationship between the statistics of the cipher text and the value of the encryption keys as complex as possible
- ✧ Diffusion: A technique that seeks to obscure the statistical structure of the plaintext by spreading out the influence of each individual plaintext digit over many cipher text digits
- ✧ The Exclusive OR (XOR) operation used by the Feistel function is a common building block of cryptography. It is a binary operation, i.e. it operates on two single bits of data at a time.

DES OPERATION

- ✚ Feistel Cipher: DES is based on the Feistel block cipher, called LUCIFER
- ✚ DES uses 16 rounds of the Feistel structure, using a different key for each round
- ✚ The idea is to partition the input block into two halves, $l_{(i-1)}$ and $r_{(i-1)}$, use only $r_{(i-1)}$ in each round i (part) of the cipher
- ✚ The function f incorporates one stage of the S-P network, controlled by part of the key $k_{(i)}$ known as the i^{th} sub key
- ✚ DES Desired Effects
 - ☞ Avalanche Effect: A small change in the plaintext gives rise to a large change in the ciphertext and Changing one input bit results in changes of approx. half the output bits
 - ☞ Completeness Effect: Each output bit is a complex function of all the input bits

DES BASICS

- ✚ DES uses the two basic techniques of cryptography – confusion and diffusion
- ✚ At the simplest level, diffusion is achieved through numerous permutations and confusions is achieved through the XOR operation
- ✚ Encryption Elements:
 - ☞ Permutation Functions (Initial and Final Permutation)
 - ☞ Expansion Permutation
 - ☞ Permuted Choices (1 and 2)
 - ☞ Substitution Functions (6 S Functions with 48 bit)
 - ☞ Data and Key
- ✚ The 64-bit plaintext passes through an initial permutation (IP) that rearranges the bits to produce the permuted input

DES BASICS

- ✧ The 64-bit block being enciphered is broken into two halves
- ✧ The right half goes through one DES round, and the result becomes the new left half
- ✧ The old left half becomes the new right half, and will go through one round in the next round
- ✧ This goes on for 16 rounds, but after the last round the left and right halves are not swapped, so that the result of the 16th round becomes the final right half, and the result of the 15th round is the final left half
- ✧ These phases consisting of 16 rounds of the same function (f) which involves both permutation and substitution
- ✧ Function f can be described as: $L_{(i)} = R_{(i-1)}$, $R_{(i)} = L_{(i-1)} \text{ XOR } P(S(E(R_{(i-1)} \text{ XOR } K_{(i)}))$
- ✧ Finally, the output is passed through an inverse permutation (IP^{-1}), to produce the 64-bit cipher text

DES ALGORITHM STEPS

1. Put the 64-bit plain text block over to an initial permutation (IP) function
 2. Create two halves of the permuted block and refer them as L_0 (Left Side Plain Text) and R_0 (Right Side Plain Text)
 3. Each L_0 and R_0 goes through 16 rounds of the encryption process
 4. Finally, rejoin L_0 and R_0 and apply Final Permutation (FP) on the newly combined block
 5. The result of this process produces the desired 64-bit ciphertext
- *The encryption process step (step 3, above) is further broken down into five stages:*
 1. Key transformation: Converts the primary key (input key) to an intermediate key
 2. Expansion permutation: Transform 32 bit length half to 48 bit length
 3. S-Box permutation: Carries out the real mixing (confusion)
 4. P-Box permutation: Carries out the diffusion or the transposition
 5. XOR and Swap: Logical operation in order to encrypt data

PROCESSING THE KEY

- ✚ There are 16 rounds of encryption in the algorithm, and a different key is used for each round.
 - ✚ Bits are labeled from 1 to 64 starting from the most significant bit and going to the least significant bit.
1. Get the 64 bit key
 2. Calculate the key schedule
 1. Perform Permuted Choice 1 on the 64 bit key

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

PROCESSING THE KEY (CONT.)

- Split the permuted key into two halves as $C[0]$ and $D[0]$, the Left and Right halves respectively
- Calculate the 16 sub keys based on the Left-Shifted Operation, start with $i = 1$ and the Number of shifts per iteration are given below:

Iteration	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Left Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

- Concatenation $C[i]D[i]$ and apply Permutation Choice 2. This will yield $K[i]$, which is 48 bits long.

14	17	11	24	1	5
3	28	15	6	21	10
32	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

- Loop back until $K[16]$ has been calculated

PROCESSING THE 64 BIT DATA BLOCK

1. Transpose the bits in the 64-block based on Initial Permutation

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

2. Split the result into two halves: L_0 - Left Side Plain text (1-32 bits) and R_0 - Right Side Plain text (33-64 bits)
3. The resulting halves undergo 16 rounds of encryption in each round
 1. Apply Expansion Permutation
 2. Apply XOR operation
 3. Apply S-Box Operation
 4. Apply P-Box Permutation

PROCESSING THE 64 BIT DATA BLOCK (CONT.)

4. Add the left half with the result from the above step and store it in the right plain text
5. Store the initial right plain text in the left plain text
6. These halves are inputs for the next round. Remember that there are different keys for each round.
7. After the 16 rounds of encryption, swap the left plain text and the right plain text.
8. Finally, apply the inverse permutation (inverse of the initial permutation), and the ciphertext will be generated.

• Expansion (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

• Permutation P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

• Final Permutation

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

PROCESSING THE 64 BIT DATA BLOCK (CONT.)

S[5]

Substitution Box 1 (S[1])

14 4 13 1 2 15 11 8 3 10 6 12 5 9 0 7
 0 15 7 4 14 2 13 11 0 6 12 11 9 5 3 8
 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0
 15 12 8 2 4 9 1 7 5 11 3 14 10 0 6 13

S[2]

15 1 8 14 6 11 3 4 9 7 2 13 12 0 5 10
 3 13 4 7 15 2 8 14 12 0 11 0 6 9 11 5
 0 14 7 11 10 4 13 1 5 8 12 6 9 3 2 15
 13 8 10 1 3 15 4 2 11 6 7 12 0 5 14 9

S[3]

10 0 9 14 6 3 15 5 1 13 12 7 11 4 2 8
 13 7 0 9 3 4 6 10 2 8 5 14 12 11 15 1
 13 6 4 9 8 15 3 0 11 1 2 12 5 10 14 7
 1 10 13 0 6 9 8 7 4 15 14 3 11 5 2 12

S[4]

7 13 14 3 0 6 9 10 1 2 8 5 11 12 4 15
 13 8 11 5 6 15 0 3 4 7 2 12 11 10 14 9
 10 6 9 0 12 11 7 13 15 1 3 14 5 2 8 4
 3 15 0 6 10 11 13 8 9 4 5 11 12 7 2 14

2 12 4 1 7 10 11 6 8 5 3 15 13 0 14 9
 14 11 2 12 4 7 13 1 5 0 15 10 3 9 8 6
 4 2 1 11 10 13 7 8 15 9 12 5 6 3 0 14
 11 8 12 7 1 14 2 13 6 15 0 9 10 4 5 3

S[6]

12 1 10 15 9 2 6 8 0 13 3 4 14 7 5 11
 10 15 4 2 7 12 9 5 6 1 13 14 0 11 3 8
 9 14 15 5 2 8 12 3 7 0 4 10 1 13 11 6
 4 3 2 12 9 5 15 10 11 14 1 7 6 0 8 13

S[7]

4 11 2 14 15 0 8 13 3 12 9 7 5 10 6 1
 13 0 11 7 4 9 1 10 14 3 5 12 2 15 8 6
 1 4 11 13 12 3 7 14 10 15 6 8 0 5 9 2
 6 11 13 8 1 4 10 7 9 5 0 15 14 2 3 12

S[8]

13 2 8 4 6 15 11 1 10 9 3 14 5 0 12 7
 1 15 13 8 10 3 7 4 12 5 6 11 0 14 9 2
 7 11 4 1 9 12 14 2 0 6 10 13 15 3 5 8
 2 1 14 7 4 10 8 13 15 12 9 0 3 5 6 11

PROCESSING THE 64 BIT DATA BLOCK (CONT.)

3. Apply the 16 sub keys to the data block (Step 3 – The Encryption Process Explained)
 1. Expand the 32-bit $R[i-1]$ into 48 bits according to the bit-selection function (Expansion (E)).
 2. Exclusive-or $E(R[i-1])$ with $K[i]$
 3. Break the result of $E(R[i-1]) \text{ xor } K[i]$ into 8 six bit blocks. Bits 1-6 are $B[1]$, bits 7-12 are $B[2]$, and so on with bits 43-48 being $B[8]$.
 4. Substitute the values found in the S-boxes for all $B[j]$. Start with $j = 1$
 5. Permute the concatenation of $B[1]$ through $B[8]$ as based on Permutation P

EXAMPLE

✧ Let the plain text Message, M: 0123456789ABCDEF => assume this message is in hexadecimal format

✧ And let the key, K = 133457799BBCDFF1 in hexadecimal format

✧ *Processing the key*

1. Get the key => we have already the key

K = 0001 0011 0011 0100 0101 0111 0111 1001 1001 1011 1011 1100 1101 1111 1111 0001

2. Performing the key Schedule

1. Perform permutation (Permuted Choice 1) : The resulting 56 bit length key is:

K+ = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111

2. split this key into halves:

$C_0 = 1111000 0110011 0010101 0101111$ $D_0 = 0101010 1011001 1001111 0001111$

3. Apply Left-Shifted operations

$C1 = 1110000110011001010101011111$ $D1 = 1010101011001100111100011110$

$C2 = 1100001100110010101010111111$ $D2 = 0101010110011001111000111101$

$C3 = 0000110011001010101011111111$ $D3 = 0101011001100111100011110101$

$C4 = 0011001100101010101111111100$ $D4 = 0101100110011110001111010101$

EXAMPLE (CONT.)

4. Concatenate C_i and D_i , and apply Permutation Choice 2 (Generating Sub-keys)

$$K_1 = 000110\ 110000\ 001011\ 101111\ 111111\ 000111\ 000001\ 110010$$

$$K_2 = 011110\ 011010\ 111011\ 011001\ 110110\ 111100\ 100111\ 100101$$

$$K_3 = 010101\ 011111\ 110010\ 001010\ 010000\ 101100\ 111110\ 011001$$

$$K_4 = 011100\ 101010\ 110111\ 010110\ 110110\ 110011\ 010100\ 011101$$

5. Loop Back until you generate all 16 sub keys

Processing the 64 bit data block

1. Get the 64 bit data block: 0123456789ABCDEF

$$M = 0000\ 0001\ 0010\ 0011\ 0100\ 0101\ 0110\ 0111\ 1000\ 1001\ 1010\ 1011\ 1100\ 1101\ 1110\ 1111$$

2. Apply Initial Permutation – IP

$$IP = 1100\ 1100\ 0000\ 0000\ 1100\ 1100\ 1111\ 1111\ 1111\ 0000\ 1010\ 1010\ 1111\ 0000\ 1010\ 1010$$

3. Split the result into two halves

$$L_0 = 1100\ 1100\ 0000\ 0000\ 1100\ 1100\ 1111\ 1111$$

$$R_0 = 1111\ 0000\ 1010\ 1010\ 1111\ 0000\ 1010\ 1010$$

4. Start the actual encryption process

EXAMPLE (CONT.)

1. Apply Expansion permutation:

$$R_0 = 1111\ 0000\ 1010\ 1010\ 1111\ 0000\ 1010\ 1010$$

$$E(R_0) = 011110\ 100001\ 010101\ 010101\ 011110\ 100001\ 010101\ 010101$$

2. Apply XOR Operation: Exclusive-or $E(R[i-1])$ with $K[i]$

$$K_1 = 000110\ 110000\ 001011\ 101111\ 111111\ 000111\ 000001\ 110010$$

$$E(R_0) = 011110\ 100001\ 010101\ 010101\ 011110\ 100001\ 010101\ 010101$$

$$K_1 \oplus E(R_0) = 011000\ 010001\ 011110\ 111010\ 100001\ 100110\ 010100\ 100111$$

3. Break the result into 6 bits of 8 blocks: Bits 1-6 are $B[1]$, bits 7-12 are $B[2]$, and so on with bits 43-48 being $B[8]$

4. Substitute the values found in the S-boxes for all $B[j]$. Start with $j=1$ and all values in the S-boxes should be considered 4 bits wide

$$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8) = 0101\ 1100\ 1000\ 0010\ 1011\ 0101\ 1001\ 0111$$

5. Permute the result using permutation table P

$$f = 0010\ 0011\ 0100\ 1010\ 1010\ 1001\ 1011\ 1011$$

6. Apply the operation: $R_1 = L_0 + f(R_0, K_1)$

$$= 1100\ 1100\ 0000\ 0000\ 1100\ 1100\ 1111\ 1111$$

$$+ 0010\ 0011\ 0100\ 1010\ 1010\ 1001\ 1011\ 1011$$

$$= 1110\ 1111\ 0100\ 1010\ 0110\ 0101\ 0100\ 0100 \Rightarrow R_1$$

7. In the next round, we will have $L_2 = R_1$, which is the block we just calculated, and then we must calculate $R_2 = L_1 + f(R_1, K_2)$, and so on for 16 rounds.

EXAMPLE (CONT.)

- ✚ If we process all 16 blocks using the method defined previously, we get, on the 16th round,
$$L16 = 0100\ 0011\ 0100\ 0010\ 0011\ 0010\ 0011\ 0100$$
$$R16 = 0000\ 1010\ 0100\ 1100\ 1101\ 1001\ 1001\ 0101$$
- ✚ We reverse the order of these two blocks and apply the final permutation to
$$R16L16 = 00001010\ 01001100\ 11011001\ 10010101\ 01000011\ 01000010\ 00110010$$
$$00110100$$
- ✚ $IP^{-1} = 10000101\ 11101000\ 00010011\ 01010100\ 00001111\ 00001010\ 10110100\ 00000101$,
which in hexadecimal format is 85E813540F0AB405.
- ✚ This is the encrypted form of $M = 0123456789ABCDEF$: namely, $C = 85E813540F0AB405$
- ✚ Decryption is simply the inverse of encryption, following the same steps as above, but reversing the order in which the sub keys are applied

DOUBLE DES

- ✧ The simplest form of double DES encryption has two encryption stages and two keys
- ✧ Given a plaintext p and two encryption keys k_1 and k_2 then the ciphertext C is generated as : $C = E_{K_2}(E_{K_1}(p))$
- ✧ Decryption requires that the keys be applied in reverse order: $P = D_{K_1}(D_{K_2}(C))$
- ✧ The scheme apparently involves a key length of 56×2 bits, resulting in a dramatic increase in cryptographic strength
- ✧ The double DES encryption only doubles the work for the attacker

TRIPLE DES

- ✚ Referred as EDE (Encryption Decryption Encryption), using two keys and applying them in three operations adds apparent strength
- ✚ Procedure: $C = EK_1(DK_2(EK_1(p)))$, that is, you encrypt with one key, decrypt with the second key and encrypt with the first key again
- ✚ It only doubles the effective key length, But a 112-bit effective key length is quite strong and it is effective against all feasible known attacks
- ✚ A straightforward implementation would be: $C = EK_1(EK_2(EK_1(P)))$ but in practice: $C = EK_1(DK_2(EK_1(P)))$
- ✚ If $K_1 = K_2$, then $3DES = 1DES$. Thus, a 3DES software can be used as a single-DES and No current known practical attacks
- ✚ Triple DES is a symmetric key-block cipher which applies the DES cipher in triplicate. It encrypts with the first key (k_1), decrypts using the second key (k_2), then encrypts with the third key (k_3). There is also a two-key variant, where k_1 and k_3 are the same keys.

DES CONCLUSION

- ✧ DES broken by exhaustive search in 1997
- ✧ DES's dominance came to an end in 2002, when the Advanced
- ✧ Encryption Standard (AES) replaced the DES encryption algorithm as the accepted standard, following a public competition to find a replacement.
- ✧ The NIST officially withdrew FIPS 46-3 (the 1999 reaffirmation) in May 2005, although Triple DES (3DES), remains approved for sensitive government information through 2030.
- ✧ The NIST had to replace the DES algorithm because its 56-bit key lengths were too small, considering the increased processing power of newer computers.
- ✧ DES found itself a victim of the ongoing technological advances in computing. It reached a point where 56-bit was no longer good enough to handle the new challenges to encryption.

ADVANCED ENCRYPTION STANDARD - AES

- ✧ A clear replacement for DES was needed: have theoretical attacks that can break it and have demonstrated exhaustive key search attacks
- ✧ In 1997, NIST began the process of choosing a replacement for DES and called it the Advanced Encryption Standard
- ✧ In 2000 NIST adopts Rijndael cipher (by Rijmen and Daemen) as AES to replace DES
- ✧ Requirements: block length of 128 bits data, key lengths of 128, 192, and 256 bits.
- ✧ AES was adopted for use by the US government in December 2001 and became the Federal Information Processing standard.
- ✧ An iterative rather than Feistel cipher: processes data as block of 4 columns of 4 bytes and operates on entire data block in every round
- ✧ An iterated cipher, with 10, 12, or 14 rounds.
- ✧ Rijndael allows various block lengths. But AES allows only one block size: 128 bits.

ADVANCED ENCRYPTION STANDARD - AES

✧ Overview of Rijndael Algorithm

✧ Designed to have:

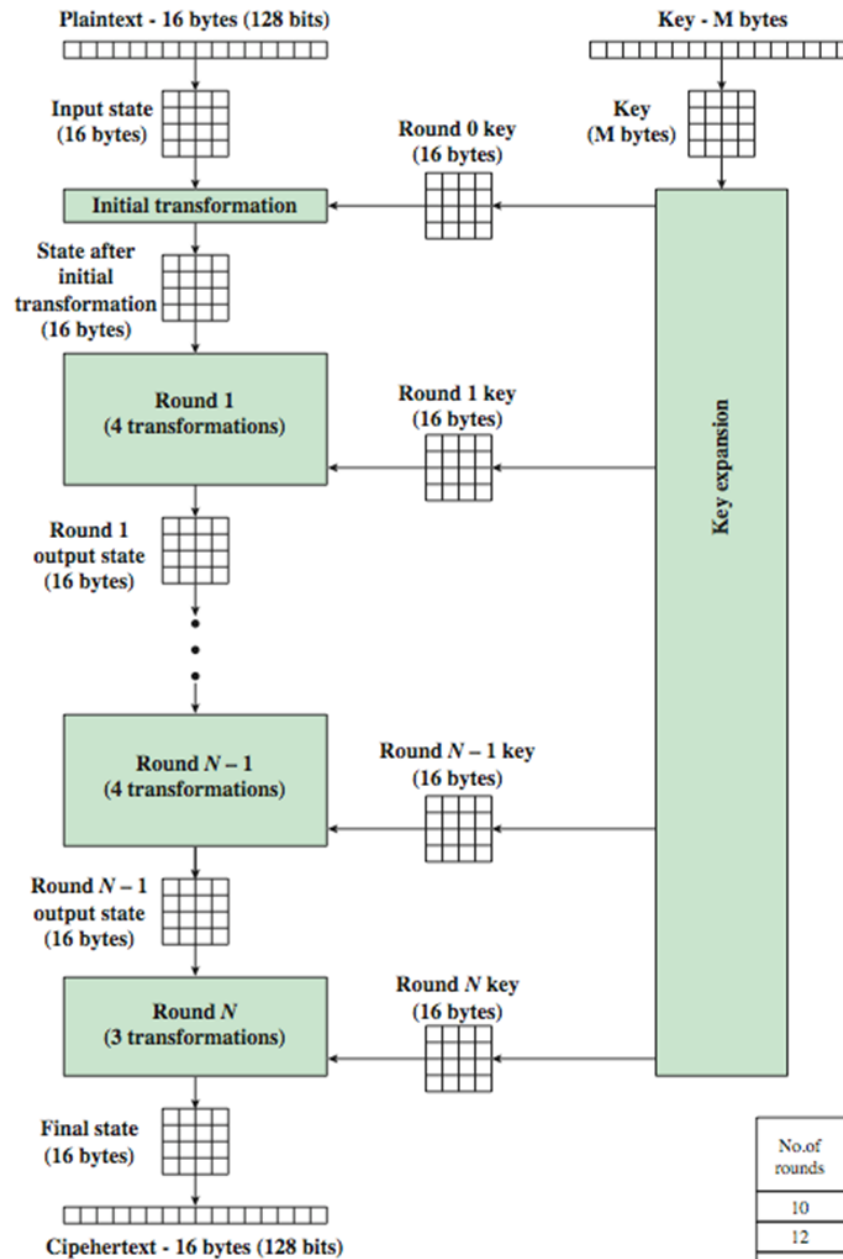
- ✧ Resistance against known attacks
- ✧ Speed and code compactness on many CPUs
- ✧ Design simplicity

✧ Although it has a strong mathematical application, it primarily uses substitution, transposition and the shift, exclusive OR and addition operations.

✧ Each cycle (round) consists of four steps:

- ✧ **Byte Substitution**:- uses a substitution box structure similar to DES
- ✧ **Shift Rows (transposition step)**:- for 128 and 192-bit block size, row n is shifted left circular (n-1) bytes: for 256-bit blocks, row 2 is shifted 1 byte and rows 3 and 4 are shifted 3 and 4 bytes respectively.
- ✧ **Mix Columns**:- this step involves shifting left and exclusive-ORing bits with themselves.
- ✧ **Add Sub keys**:- Here, a portion of the key unique to this cycle is exclusive-ORed with the cycle result

AES ENCRYPTION PROCESS



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

AES STRUCTURE

- ✚ Data block of 4 columns of 4 bytes is state
- ✚ Key is expanded to array of words
- ✚ Has 9/11/13 rounds in which state undergoes:
 - ✚ Byte substitution (1 S-box used on every byte)
 - ✚ Shift rows (permute bytes between groups/columns)
 - ✚ Mix columns (subs using matrix multiply of groups)
 - ✚ Add round key (XOR state with key material)
 - ✚ View as alternating XOR key & scramble data bytes
- ✚ Initial XOR key material & incomplete last round
- ✚ With fast XOR & table lookup implementation

COMPARISON OF DES AND AES

	DES	AES
Date	1976	1999
Block Size	64-bits	128-bits
Key length	56-bits(effective Key length)	128,192,256 [and possibly more] - bits
Encryption Primitives	Substitution, Permutation	Substitution, shift, bit mixing
Cryptographic Primitives	Confusion, Diffusion	Confusion, Diffusion
Design	Open	Open
Design Rationale	Closed	Open
Selection Process	Secret	Secret, but accepted open public comment
Source	IBM, enhanced by NSA	Independent Dutch cryptographers

AES - CONCLUSION

- ✧ An iterative rather than Feistel cipher
- ✧ Key expanded into array of 32-bit words
 - ☞ Four words form round key in each round
- ✧ 4 different stages are used as shown
- ✧ Has a simple structure
- ✧ Only AddRoundKey uses key
- ✧ AddRoundKey a form of Vernam cipher
- ✧ Each stage is easily reversible
- ✧ Decryption uses keys in reverse order
- ✧ Decryption does recover plaintext
- ✧ Final round has only 3 stages